

Artificial Intelligence (AI-2002)

Lecture 6: Solving Problems by Searching

Mr. Saif ur Rehman (Lecturer – AI & DS)

saifurrehman@nu.edu.pk



SOLVING PROBLEMS BY SEARCHING

- An agent that plans a sequence of actions to achieve a goal state when the correct action isn't immediately clear, utilizing a computational process called search.
- Problem-solving agents use atomic representations, where world states are treated as whole entities with no internal structure visible to the algorithms.
- We will focus on search algorithms for simple environments that are episodic, single-agent, fully observable, deterministic, static, discrete, and known. It also distinguishes between informed (with goal estimates) and uninformed (without goal estimates) algorithms.

Problem-Solving Agents

- The agent needs to travel from Arad to Bucharest.
- The agent sees three roads leading out of Arad: one to Sibiu, one to Timisoara, and one to Zerind.
- None of these roads lead directly to the goal (Bucharest).
- If the agent doesn't know the geography of Romania, it will not know which road to choose.
- Without additional information (in an unknown environment), the agent might randomly choose one of the roads.

Problem-Solving Agents

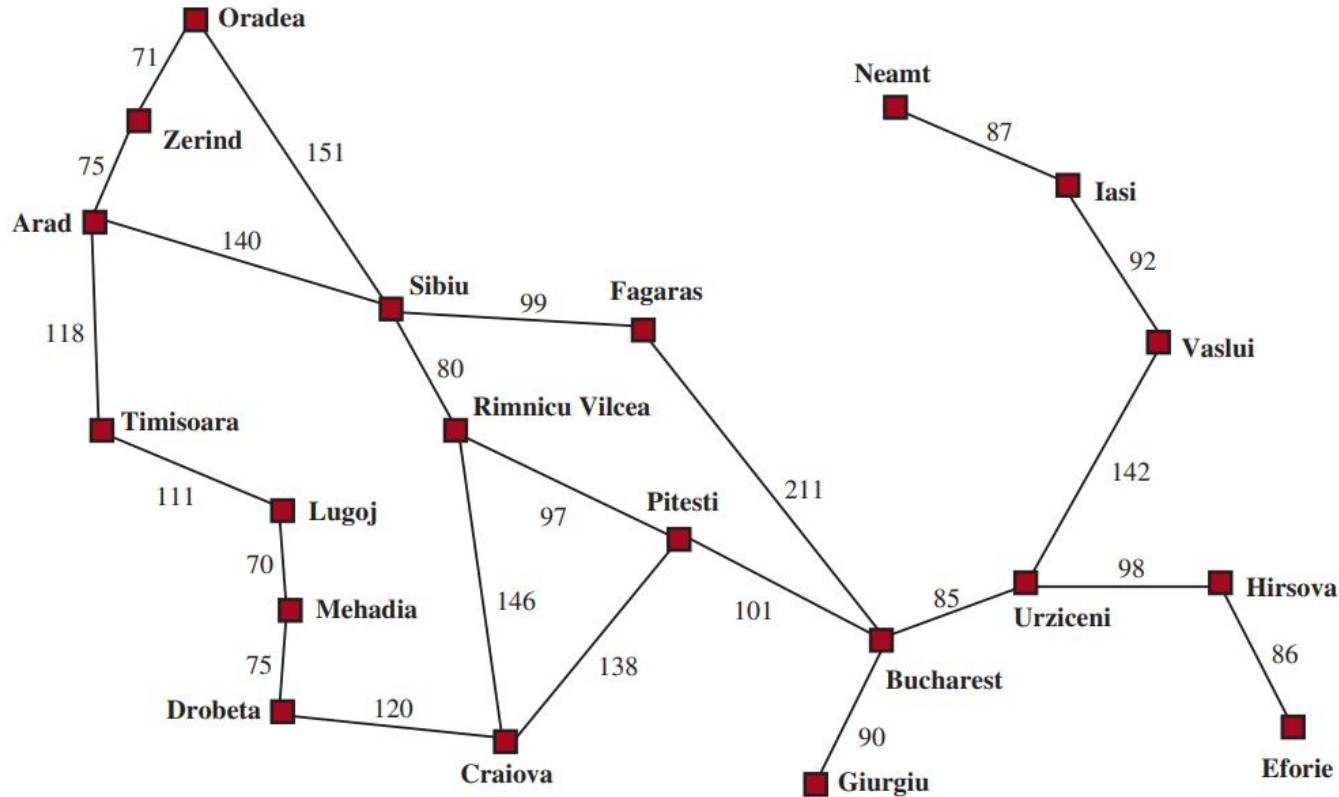


Figure 3.1 A simplified road map of part of Romania, with road distances in miles.

Problem-Solving Agents

- We will assume our agents always have access to information about the world. With that information, the agent can follow this four-phase problem-solving process:
 1. **Goal formulation:** The agent sets the goal to reach Bucharest, organizing its behavior and focusing on actions that help achieve this goal.
 2. **Problem formulation:** The agent creates a model of the world, describing states and actions needed to reach the goal. For this agent, the model focuses on traveling from one city to an adjacent city, with the current city being the only changing state.

Problem-Solving Agents

3. **Search:** The agent simulates possible sequences of actions in its model to find a path that leads to the goal. It may try multiple sequences before finding a solution or discovering that no solution exists.

4. **Execution:** Once the agent finds a solution, such as traveling from Arad to Bucharest, it can execute the actions step by step in the real world.

Problem-Solving Agents

- In a fully observable, deterministic, known environment, the solution is always a fixed sequence of actions (e.g., drive from Arad to Sibiu, then Fagaras, then Bucharest).
- The sequence of actions remains the same every time.

Search problems and solutions

A search problem can be defined formally as follows:

- A set of possible states that the environment can be in. We call this the state space.
- The initial state that the agent starts in. For example: Arad
- A set of one or more goal states.
- The actions available to the agent. Given a state s , $ACTIONS(s)$ returns a finite set of Action actions that can be executed in s .
- An example: $ACTIONS(Arad) = \{ToSibiu, ToTimisoara, ToZerind\}$.

Search problems and solutions

- A transition model, which describes what each action does. $\text{RESULT}(s, a)$ returns the Transition model state that results from doing action a in state s . For example, $\text{RESULT}(\text{Arad}, \text{ToZerind}) = \text{Zerind}$.
- An action cost function, denoted by $\text{ACTION-COST}(s, a, s')$ that gives the numeric cost of applying action a in state s to reach state s' .

Search problems and solutions

- A sequence of actions forms a path, and a solution is a path from the initial state to a goal Path state.
- We assume that action costs are additive; that is, the total cost of a path is the sum of the individual action costs.
- An optimal solution has the lowest path cost among all solutions.
- We assume that all action costs will be positive, to avoid certain complications.
- The state space can be represented as a graph in which the vertices are states and the Graph directed edges between them are actions.

Formulating problems

- The problem formulation we've used to describe the journey from Arad to Bucharest is a model—a simplified, abstract mathematical description of the situation.
- Compare the simple atomic state description Arad to an actual cross-country trip where unnecessary are left out of our model.
- The process of removing detail from a representation is called abstraction. A good problem formulation has the right level of detail

Example Problems

- The problem-solving approach has been applied to a vast array of task environments. We list some of the best known here, distinguishing between standardized and real-world problems.
- A **standardized problem** is intended to illustrate or exercise various problem-solving methods. It can be given a concise, exact description and hence is suitable as a benchmark for researchers to compare the performance of algorithms.
- A **real-world problem**, such as robot navigation, is one whose solutions people actually use, and whose formulation is idiosyncratic, not standardized, because, for example, each robot has different sensors that produce different data.

Example Problems

Define the problem for the Vacuum World.

- States: a vacuum environment with n cells has $n \cdot 2^n$ states.
- Initial state: Any state can be designated as the initial state.
- Actions: In the two-cell world we defined three actions: Suck, move Left, and move Right.
- Transition model: Suck removes any dirt from the agent's cell; Left moves the agent left and right moves the agent right.

Example Problems

Define the problem for the Vacuum World.

- States: a vacuum environment with n cells has $n \cdot 2^n$ states.
- Initial state: Any state can be designated as the initial state.
- Actions: In the two-cell world we defined three actions: Suck, move Left, and move Right.
- Transition model: Suck removes any dirt from the agent's cell; Left moves the agent left and right moves the agent right.
- Goal states: The states in which every cell is clean.
- Action cost: Each action costs 1.

Example Problems

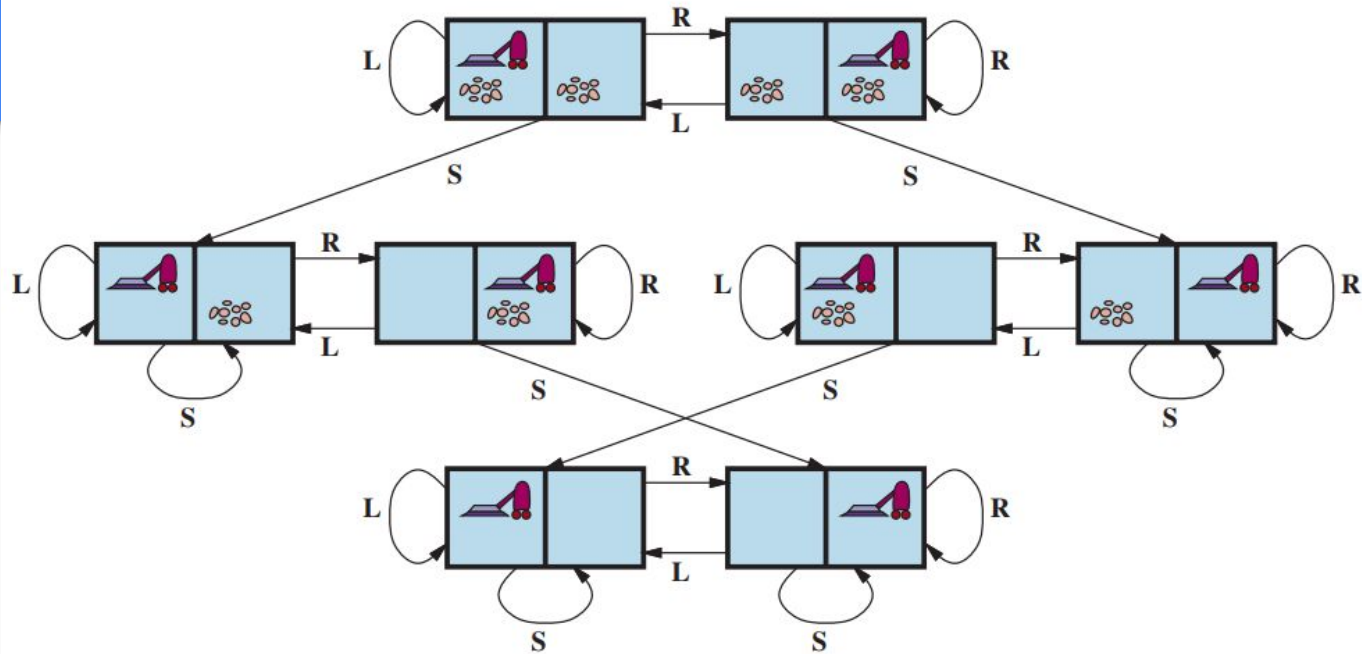


Figure 3.2 The state-space graph for the two-cell vacuum world. There are 8 states and three actions for each state: L = *Left*, R = *Right*, S = *Suck*.

Example Problems

Define the problem for the sokoban puzzle:

- For a world with n non-obstacle cells and b boxes, there are states;
d $n \times n! / (b!(n-b)!)$ states an 8×8 grid with a dozen boxes, there are over 200 trillion states.
- **Initial state:**
- **Actions:**
- **Transition model:**
- **Goal state:**
- **Action cost:**

Example Problems

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

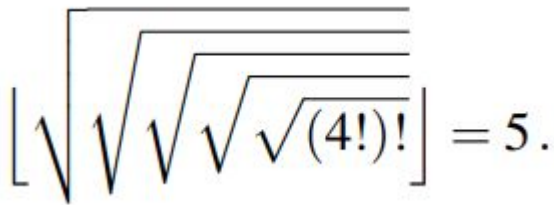
Figure 3.3 A typical instance of the 8-puzzle.

Example Problems

- **States:** A state description specifies the location of each of the tiles.
- **Initial state:** Any state can be designated as the initial state. Note that a parity property partitions the state space—any given goal can be reached from exactly half of the possible initial states (see Exercise 3.PART).
- **Actions:** While in the physical world it is a tile that slides, the simplest way of describing an action is to think of the blank space moving *Left*, *Right*, *Up*, or *Down*. If the blank is at an edge or corner then not all actions will be applicable.
- **Transition model:** Maps a state and action to a resulting state; for example, if we apply *Left* to the start state in Figure 3.3, the resulting state has the 5 and the blank switched.
- **Goal state:** Although any state could be the goal, we typically specify a state with the numbers in order, as in Figure 3.3.
- **Action cost:** Each action costs 1.

Example Problems

Our final standardized problem was devised by Donald Knuth (1964) and illustrates how infinite state spaces can arise. Knuth conjectured that starting with the number 4, a sequence of square root, floor, and factorial operations can reach any desired positive integer. For example, we can reach 5 from 4 as follows:


$$\left[\sqrt{\sqrt{\sqrt{\sqrt{(4!)!}}} \right] = 5.$$

- **Initial state:**
- **Actions:**
- **Transition model:**
- **Goal state:**
- **Action cost:**

Example Problems

$$\lfloor \sqrt{\sqrt{\sqrt{\sqrt{\sqrt{\sqrt{(4!)!}}}}} \rfloor = 5.$$

The problem definition is simple:

- **States:** Positive real numbers.
- **Initial state:** 4.
- **Actions:** Apply square root, floor, or factorial operation (factorial for integers only).
- **Transition model:** As given by the mathematical definitions of the operations.
- **Goal state:** The desired positive integer.
- **Action cost:** Each action costs 1.

Example Problems

Define the problem for the route-finding problem:

- **Initial state:**
- **Actions:**
- **Transition model:**
- **Goal state:**
- **Action cost:**

Example Problems

Define the problem for the route-finding problem:

- **Initial state:**
- **Actions:**
- **Transition model:**
- **Goal state:**
- **Action cost:**

Example Problems

- **States:** Each state obviously includes a location (e.g., an airport) and the current time. Furthermore, because the cost of an action (a flight segment) may depend on previous segments, their fare bases, and their status as domestic or international, the state must record extra information about these “historical” aspects.
- **Initial state:** The user’s home airport.
- **Actions:** Take any flight from the current location, in any seat class, leaving after the current time, leaving enough time for within-airport transfer if needed.
- **Transition model:** The state resulting from taking a flight will have the flight’s destination as the new location and the flight’s arrival time as the new time.
- **Goal state:** A destination city. Sometimes the goal can be more complex, such as “arrive at the destination on a nonstop flight.”
- **Action cost:** A combination of monetary cost, waiting time, flight time, customs and immigration procedures, seat quality, time of day, type of airplane, frequent-flyer reward points, and so on.